```python
# Importing pandas library for data handling
# Loading the customer master dataset into a DataFrame
# Displaying the first 5 rows using head

import pandas as pd
df = pd.read_csv(r"/content/Customer_Master_Data.csv")
df.head()
```


{"summary":"{\n \"name\": \"df\",\n \"rows\": 1000,\n \"fields\": [\n {\n \"column\": \"CustomerID\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 1000,\n \"samples\": [\n \"CUST10521\",\n \"CUST10737\",\n \"CUST10740\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Name\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 993,\n \"samples\": [\n \"Vanya Rattan\",\n \"Shanaya Jhaveri\",\n \"Raunak Choudhry\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Email\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 1000,\n \"samples\": [\n \"btara@bora.net\",\n \"ckeer@hotmail.com\",\n \"elarajan@rao-kaur.com\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Gender\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 3,\n \"samples\": [\n \"Male\",\n \"Female\",\n \"Not Disclosed\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Age\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 16,\n \"min\": 18,\n \"max\": 75,\n \"num_unique_values\": 58,\n \"samples\": [\n 54,\n 59,\n 24\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"City\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 10,\n \"samples\": [\n \"Chennai\",\n \"Kolkata\",\n \"Ahmedabad\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"MaritalStatus\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 4,\n \"samples\": [\n \"Married\",\n \"Single\",\n \"Divorced\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"NumChildren\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1,\n \"min\": 0,\n \"max\": 4,\n \"num_unique_values\": 5,\n \"samples\": [\n 2,\n 4,\n 3\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"JoinDate\",\n \"properties\":

{\n   \"dtype\": \"object\",\n   \"num_unique_values\": 727,\n   \"samples\": [\n   \"2021-03-30\",\n   \"2021-10-18\",\n   \"2020-10-04\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
# Importing pandas library for data handling
# Loading the customer master dataset into a DataFrame
# Displaying the first 5 rows using head

df1 = pd.read_csv(r"/content/Customer_Transactions.csv")
df.head()
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 1000,\n \"fields\": [\n {\n   \"column\": \"CustomerID\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 1000,\n   \"samples\": [\n   \"CUST10521\",\n   \"CUST10737\",\n   \"CUST10740\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"Name\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 993,\n   \"samples\": [\n   \"Vanya Rattan\",\n   \"Shanaya Jhaveri\",\n   \"Raunak Choudhry\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"Email\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 1000,\n   \"samples\": [\n   \"btara@bora.net\",\n   \"ckeer@hotmail.com\",\n   \"elarajan@rao-kaur.com\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"Gender\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 3,\n   \"samples\": [\n   \"Male\",\n   \"Female\",\n   \"Not Disclosed\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"Age\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 16,\n   \"min\": 18,\n   \"max\": 75,\n   \"num_unique_values\": 58,\n   \"samples\": [\n   54,\n   59,\n   24\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"City\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 10,\n   \"samples\": [\n   \"Chennai\",\n   \"Kolkata\",\n   \"Ahmedabad\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"MaritalStatus\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 4,\n   \"samples\": [\n   \"Married\",\n   \"Single\",\n   \"Divorced\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n },\n {\n   \"column\": \"NumChildren\",\n   \"properties\": {\n   \"dtype\":

\"number\",\n   \"std\": 1,\n   \"min\": 0,\n   \"max\": 4,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   2,\n   4,\n   3\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"JoinDate\",\n   \"properties\": {\n   \"dtype\": \"object\",\n   \"num_unique_values\": 727,\n   \"samples\": [\n   \"2021-03-30\",\n   \"2021-10-18\",\n   \"2020-10-04\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
df.shape          #to check the no. of rows and columns
```

```
(1000, 9)
```

```
df1.shape
```

```
(23050, 3)
```

```
df.info()        #to diplay all the info of the dataset such as columns,datatype and null values
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1000 non-null   object
 1   Name           1000 non-null   object
 2   Email          1000 non-null   object
 3   Gender         1000 non-null   object
 4   Age            1000 non-null   int64
 5   City           1000 non-null   object
 6   MaritalStatus  1000 non-null   object
 7   NumChildren    1000 non-null   int64
 8   JoinDate       1000 non-null   object
dtypes: int64(2), object(7)
memory usage: 70.4+ KB
```

```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         23050 non-null  object
 1   TransactionDate    23050 non-null  object
 2   TransactionAmount  23050 non-null  float64
dtypes: float64(1), object(2)
memory usage: 540.4+ KB
```

```
#Merging df and df1 to get joindate in the master table by using right
join on CustomerID

final_df = pd.merge(df, df1, on="CustomerID", how="right")

final_df
```


{"summary":"{\n \"name\": \"final_df\",\n \"rows\": 23050,\n
\"fields\": [\n {\n \"column\": \"CustomerID\",\n
\"properties\": {\n \"dtype\": \"category\",\n
\"num_unique_values\": 1000,\n \"samples\": [\n
\"CUST10972\",\n \"CUST10246\",\n \"CUST10578\"\n
],\n \"semantic_type\": \"\",\n \"description\": \"\"\n
}\n },\n {\n \"column\": \"Name\",\n \"properties\":
{\n \"dtype\": \"category\",\n \"num_unique_values\":
993,\n \"samples\": [\n \"Kimaya Raja\",\n
\"Hunar Ramesh\",\n \"Shalv Agrawal\"\n],\n
\"semantic_type\": \"\",\n \"description\": \"\"\n }\
n },\n {\n \"column\": \"Email\",\n \"properties\": {\
n \"dtype\": \"category\",\n \"num_unique_values\":
1000,\n \"samples\": [\n \"mangatsaanvi@kohli.com\",\n
\"itandon@yahoo.com\",\n \"kalladevansh@bose-mangal.com\"\n
],\n \"semantic_type\": \"\",\n \"description\": \"\"\n
}\n },\n {\n \"column\": \"Gender\",\n \"properties\":
{\n \"dtype\": \"category\",\n \"num_unique_values\":
3,\n \"samples\": [\n \"Female\",\n
\"Male\",\n \"Not Disclosed\"\n],\n
\"semantic_type\": \"\",\n \"description\": \"\"\n }\
n },\n {\n \"column\": \"Age\",\n \"properties\": {\n
\"dtype\": \"number\",\n \"std\": 16,\n \"min\": 18,\n
\"max\": 75,\n \"num_unique_values\": 58,\n \"samples\":
[\n 27,\n 36,\n 52\n],\n
\"semantic_type\": \"\",\n \"description\": \"\"\n }\
n },\n {\n \"column\": \"City\",\n \"properties\": {\n
\"dtype\": \"category\",\n \"num_unique_values\": 10,\n
\"samples\": [\n \"Delhi\",\n \"Lucknow\",\n
\"Chennai\"\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n },\n {\n \"column\":
\"MaritalStatus\",\n \"properties\": {\n \"dtype\":
\"category\",\n \"num_unique_values\": 4,\n \"samples\":
[\n \"Divorced\",\n \"Married\",\n
\"Widowed\"\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n },\n {\n \"column\":
\"NumChildren\",\n \"properties\": {\n \"dtype\":
\"number\",\n \"std\": 1,\n \"min\": 0,\n
\"max\": 4,\n \"num_unique_values\": 5,\n \"samples\":
[\n 4,\n 1,\n 0\n],\n
\"semantic_type\": \"\",\n \"description\": \"\"\n }\
n },\n {\n \"column\": \"JoinDate\",\n \"properties\":
{\n \"dtype\": \"object\",\n \"num_unique_values\":

727,\n   \"samples\": [\n   \"2021-02-17\",\n
\"2020-11-16\",\n   \"2024-06-12\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"TransactionDate\",\n
\"properties\": {\n   \"dtype\": \"object\",\n
\"num_unique_values\": 1713,\n   \"samples\": [\n
\"8/23/22\",\n   \"12/12/20\",\n   \"1/26/21\"\
n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"TransactionAmount\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 703.3589776601669,\n   \"min\":
3.87,\n   \"max\": 6890.19,\n   \"num_unique_values\":
21810,\n   \"samples\": [\n   219.77,\n   1328.5,\n
102.46\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"final_df"}

```
#converting Transactiondate and Joindate to date time

final_df['TransactionDate'] =
pd.to_datetime(final_df['TransactionDate'])
final_df['JoinDate'] = pd.to_datetime(final_df['JoinDate'])
```

```
/tmp/ipython-input-2986833044.py:1: UserWarning: Could not infer
format, so each element will be parsed individually, falling back to
`dateutil`. To ensure parsing is consistent and as-expected, please
specify a format.
  final_df['TransactionDate'] =
pd.to_datetime(final_df['TransactionDate'])
```

```
#To check the count of null values in each column

final_df.isnull().sum()
```

```
CustomerID           0
Name                 0
Email                0
Gender               0
Age                  0
City                 0
MaritalStatus        0
NumChildren          0
JoinDate             0
TransactionDate      0
TransactionAmount    0
dtype: int64
```

```
df['CustomerID'].is_unique    #to check whether customerID has only
unique values
```

```
True
```

```
df1["CustomerID"].isin(df["CustomerID"]).all()       #To check if all
the values of df1 are in df for CustomerID column
```

```
np.True_
```

```
# RFM applied on final_df by using groupby and agg functions

df_rfm= final_df.groupby("CustomerID").agg(monetary=
('TransactionAmount', 'sum'),
                                           frequency=
('TransactionDate','count'))
reference_date = final_df['TransactionDate'].max()
+pd.Timedelta(days=1)
last_transaction = final_df.groupby('CustomerID')
['TransactionDate'].max()
recency = (reference_date-last_transaction).dt.days
df_rfm['recency']= recency

#Providing labels to the R_score, F_score and M_score.

df_rfm['R_score'] = pd.qcut(df_rfm['recency'], q=5,
labels=[5,4,3,2,1])
df_rfm['F_score'] = pd.qcut(df_rfm['frequency'], q=5,
labels=[1,2,3,4,5])
df_rfm['M_score'] = pd.qcut(df_rfm['monetary'], q=5,
labels=[1,2,3,4,5])

df_rfm
```


```
{"summary":"{\n  \"name\": \"df_rfm\",\n  \"rows\": 1000,\n
\"fields\": [\n    {\n      \"column\": \"CustomerID\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
\"CUST10521\",\n          \"CUST10737\",\n          \"CUST10740\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"monetary\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
5622.441010112076,\n        \"min\": 5052.6900000000005,\n
\"max\": 44784.99,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          15914.19,\n          18145.49,\n
12293.19\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"frequency\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4,\n        \"min\": 6,\n
\"max\": 38,\n        \"num_unique_values\": 29,\n        \"samples\":
[\n          36,\n          33,\n          18\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"recency\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 58,\n
\"min\": 1,\n        \"max\": 524,\n        \"num_unique_values\":
184,\n        \"samples\": [\n          19,\n          33,\n
```

143\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"R_score\",\n   \"properties\": {\n   \"dtype\":
\"category\",\n   \"num_unique_values\": 5,\n   \"samples\":
[\n   3,\n   2,\n   1\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"F_score\",\n   \"properties\":
{\n   \"dtype\": \"category\",\n   \"num_unique_values\":
5,\n   \"samples\": [\n   5,\n   2,\n   4\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"M_score\",\n
\"properties\": {\n   \"dtype\": \"category\",\n
\"num_unique_values\": 5,\n   \"samples\": [\n   5,\n
1,\n   3\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"df_rfm"}

```
#As asked in the question adding new column - Segment to df_rfm (has
been converted to str to get the desired output like '453')

df_rfm['Segment'] = (df_rfm['R_score'].astype('str')+
df_rfm['F_score'].astype('str')+df_rfm['M_score'].astype('str'))

df_rfm
```

{"summary":"{\n \"name\": \"df_rfm\",\n \"rows\": 1000,\n
\"fields\": [\n   {\n   \"column\": \"CustomerID\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 1000,\n   \"samples\": [\n
\"CUST10521\",\n   \"CUST10737\",\n   \"CUST10740\"\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"monetary\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
5622.441010112076,\n   \"min\": 5052.6900000000005,\n
\"max\": 44784.99,\n   \"num_unique_values\": 1000,\n
\"samples\": [\n   15914.19,\n   18145.49,\n
12293.19\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"frequency\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 4,\n   \"min\": 6,\n
\"max\": 38,\n   \"num_unique_values\": 29,\n   \"samples\":
[\n   36,\n   33,\n   18\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"recency\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\": 58,\n
\"min\": 1,\n   \"max\": 524,\n   \"num_unique_values\":
184,\n   \"samples\": [\n   19,\n   33,\n
143\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"R_score\",\n   \"properties\": {\n   \"dtype\":

\"category\",\n   \"num_unique_values\": 5,\n   \"samples\": [\n   3,\n   2,\n   1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"F_score\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 5,\n   \"samples\": [\n   5,\n   2,\n   4\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"M_score\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 5,\n   \"samples\": [\n   5,\n   1,\n   3\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Segment\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 104,\n   \"samples\": [\n   \"223\",\n   \"154\",\n   \"521\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"df_rfm"}

```
#Assigning labels to the segment using df.loc

df_rfm['SegmentLabels'] = 'Others'    #values which will not fulfill
any condition will be marked as others

#1 Champions:
df_rfm.loc[ (df_rfm['R_score'] >=4) &
        (df_rfm['F_score'] >=4) &
        (df_rfm['M_score'] >=4),
        'SegmentLabels'] = 'Champions'

#2 Loyal Customers:
df_rfm.loc[ (df_rfm['R_score'] >=3) &
        (df_rfm['F_score'] >=4) &
        (df_rfm['M_score'] >=3),
        'SegmentLabels'] = 'Loyal Customers'

#3 Potential Loyalist
df_rfm.loc[ (df_rfm['R_score'] >=4) &
        (df_rfm['F_score'] <=3) &
        (df_rfm['M_score'] >=2),
        'SegmentLabels'] = 'Potential Loyalist'

#4 At Risk
df_rfm.loc[ (df_rfm['R_score'] <=2) &
        (df_rfm['F_score'] >=3) &
        (df_rfm['M_score'] <=2),
        'SegmentLabels'] = 'At Risk'

#5 Hibernating
df_rfm.loc[ (df_rfm['R_score'] <=2) &
```

```
        (df_rfm['F_score'] <=2) &
        (df_rfm['M_score'] <=2),
        'SegmentLabels'] = 'Hibernating'

df_rfm.head()
```


{"summary":"{\n \"name\": \"df_rfm\",\n \"rows\": 1000,\n \"fields\": [\n {\n \"column\": \"CustomerID\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 1000,\n \"samples\": [\n \"CUST10521\",\n \"CUST10737\",\n \"CUST10740\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"monetary\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 5622.441010112076,\n \"min\": 5052.6900000000005,\n \"max\": 44784.99,\n \"num_unique_values\": 1000,\n \"samples\": [\n 15914.19,\n 18145.49,\n 12293.19\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"frequency\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 4,\n \"min\": 6,\n \"max\": 38,\n \"num_unique_values\": 29,\n \"samples\": [\n 36,\n 33,\n 18\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"recency\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 58,\n \"min\": 1,\n \"max\": 524,\n \"num_unique_values\": 184,\n \"samples\": [\n 19,\n 33,\n 143\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"R_score\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 5,\n \"samples\": [\n 3,\n 2,\n 1\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"F_score\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 5,\n \"samples\": [\n 5,\n 2,\n 4\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"M_score\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 5,\n \"samples\": [\n 5,\n 1,\n 3\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Segment\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 104,\n \"samples\": [\n \"223\",\n \"154\",\n \"521\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"SegmentLabels\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 6,\n \"samples\":

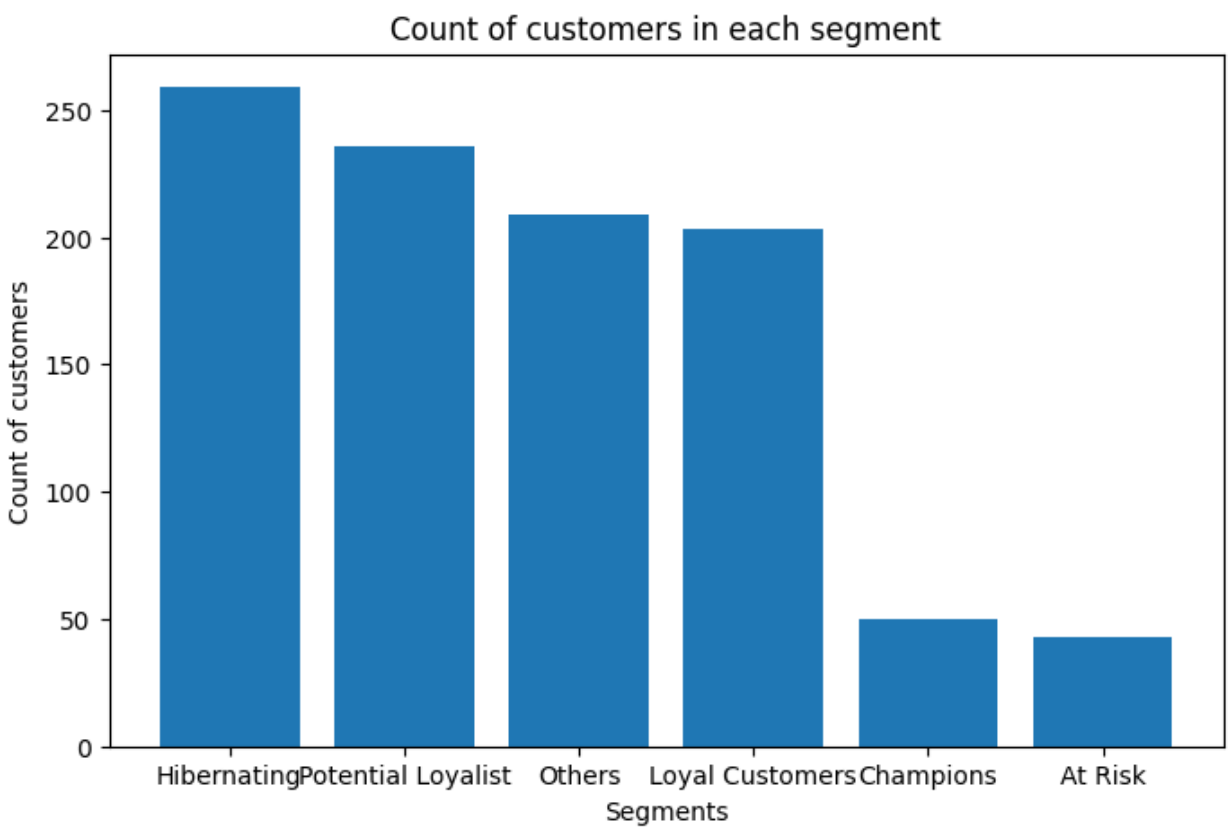
[\n   \"At Risk\",\n   \"Loyal Customers\",\n
\"Hibernating\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"df_rfm"}

```
#Bar chart for count of cutomers in each segment using matplotlib

import matplotlib.pyplot as plt
segment_counts = df_rfm['SegmentLabels'].value_counts()     #count of
customers in each segment

plt.figure(figsize=(8,5))
plt.bar(segment_counts.index, segment_counts.values)
plt.xlabel("Segments")
plt.ylabel("Count of customers")
plt.title("Count of customers in each segment")
```

```
Text(0.5, 1.0, 'Count of customers in each segment')
```

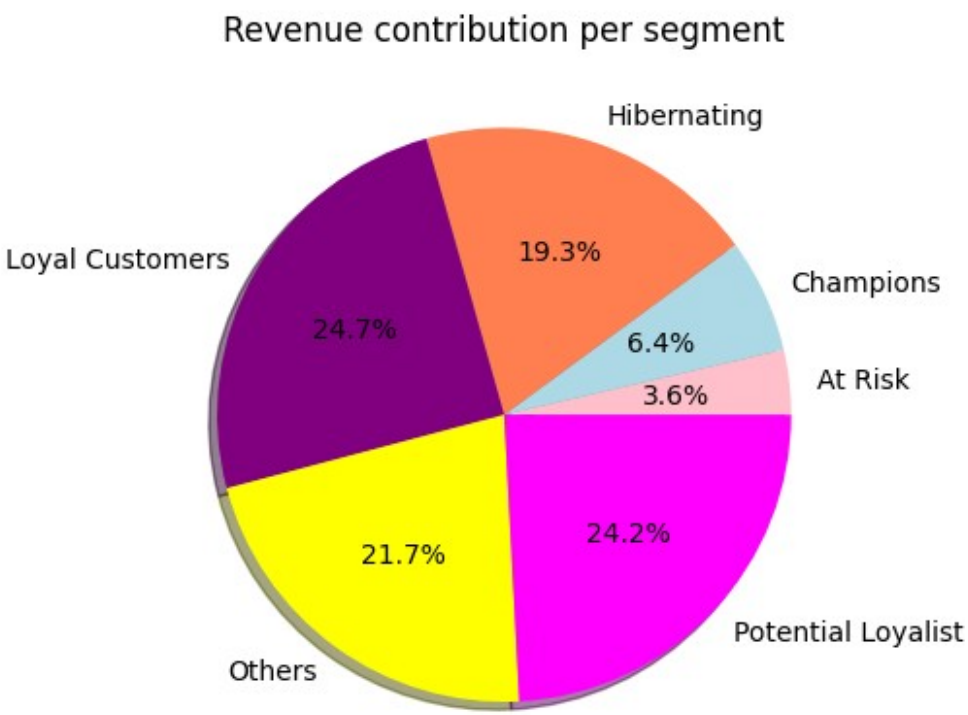



```
#Pie chart for revenue contribution per each segment

revenue_by_segment = df_rfm.groupby('SegmentLabels')['monetary'].sum()
labels = revenue_by_segment.index
color = ['pink','lightblue','coral','purple','yellow','magenta']
```

```
plt.pie(revenue_by_segment, labels=labels, autopct='%1.1f%%', colors=
color, shadow=True)
plt.title("Revenue contribution per segment")
```

```
Text(0.5, 1.0, 'Revenue contribution per segment')
```

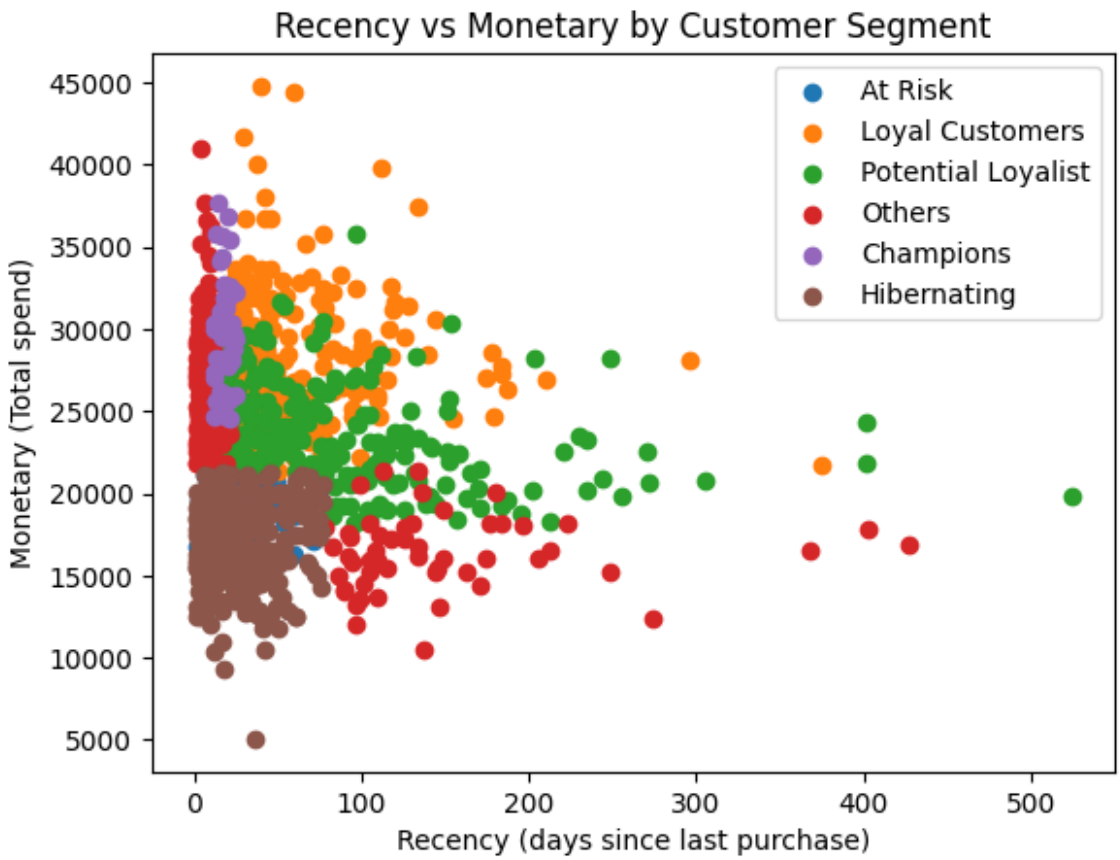



```
#Scatter plot for recency vs monetary coloured as per each segment

for segment in df_rfm['SegmentLabels'].unique():
    temp = df_rfm[df_rfm['SegmentLabels'] == segment]
    plt.scatter(temp['recency'], temp['monetary'], label=segment)

plt.xlabel('Recency (days since last purchase)')
plt.ylabel('Monetary (Total spend)')
plt.title('Recency vs Monetary by Customer Segment')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7f7f7e36ac90>
```


Recency vs Monetary by Customer Segment
At Risk
Loyal Customers
Potential Loyalist
Others
Champions
Hibernating
Monetary (Total spend)
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
100
200
300
400
500
Recency (days since last purchase)